CAS Harness — App Design Briefs (Parts 1–3)

Customer-facing interpretability app · for Claude Design · 2026-08-22

CAS Harness — Product Design Brief

For Claude Design. Customer-facing interpretability app.

---

1. What this product actually is

An app that tells you **what a language model represents internally**, using tests where **cheating is arithmetically impossible**.

That last clause is the entire product. Every other interpretability tool reports "our probe beat a baseline." A skeptic asks "was your baseline weak?" and the conversation dies. This harness constructs its test sentences so that **any word-counting method is mathematically pinned to chance** — verified in code before a model is ever loaded. When a score rises above that line, only one explanation remains.

The design must make that single idea legible in under five seconds.

---

2. Design principles (in priority order)

1. **The floor is the hero.** Every number in this app is meaningless without the line it beat. Never show a score alone. The chance line and the pinned surface baseline are not annotations — they are the primary visual.
2. **One idea per screen.** If a screen needs a paragraph to explain itself, the screen is wrong.

3. **Honest states are first-class.** `REAL`, `BOUNDED`, `WITHDRAWN`, `NOT RUN` are not error states to hide. Showing withdrawn results is the trust signal that separates this from every dashboard that only shows wins. Design them as deliberately as the wins.
4. **Numbers exact, never rounded.** `0.8604` not "86%". Rounding is a tell that the product is decorative.
5. **Progressive disclosure.** Surface: one verdict. One tap: the evidence. Two taps: the raw method and the reproduction command. Never all three at once.
6. **Silence between elements.** Generous whitespace is the sophistication. Resist filling it.

Explicitly forbidden: gradient-filled hero panels, glowing "AI" orbs, animated particle backgrounds, neural-network line art, badge clusters, marketing adjectives in the UI ("powerful", "revolutionary", "cutting-edge"), tooltips explaining obvious controls, more than two typefaces, more than one accent hue per screen.

---

3. Visual language

Reference points: Google's information density without ornament. Apple's confidence in empty space and one strong typographic hierarchy. Think Linear, Vercel dashboard, Stripe docs — not "enterprise AI platform."

- **Type:** one grotesque for UI (Inter, or system stack). One monospace for every number, hash, path, and model ID. Numbers never render in the UI face.
- **Palette:** near-black or near-white ground. One accent used *only* for "measured signal." Neutral greys carry everything else. Claim-status colours are the second and only other hue family — and they are muted, not traffic-light.
- **Status colours:** verified (calm green), bounded (amber), withdrawn (desaturated red), not-run (grey outline, no fill). Never saturated. These appear as a 6px left rule or a small dot — never as a filled pill covering a row.
- **Motion:** only to show causality — a bar growing from its floor as a run completes. No decorative transitions. Nothing loops.

- **Charts:** no chart junk. No gridlines beyond the floor line. No 3D, no gradients in data marks, no legends where direct labels fit.
- **Dark and light must both be first-class**, not an inverted afterthought.

---

4. The signature component — "The Floor"

This is the app's one memorable visual, reused everywhere.

A horizontal track. A **hard, unmissable line** at the pinned baseline (`0.500` for a two-way test). The measured score is a bar rising from that line. A thin bracket shows the 95% confidence interval.

Three states:

- Bar clears the line, interval fully above it → **signal**
- Interval crosses the line → **inconclusive**, drawn deliberately weak
- Bar at or below → **null**, drawn flat and grey, not as a failure

Under the track, one line of monospace:

```
0.8604   95% CI [0.8444, 0.8764]   floor 0.500 (pinned)
```

The word `pinned` carries the whole product claim. On tap it expands to explain: *word multisets identical, character n-gram bags identical, residual 0.000000 — a word-counter cannot exceed 0.500 here.*

Everything else in the app is a layout around this component.

---

5. Core flows

5.1 Connect a model

Two paths on one screen. No wizard, no stepper.

Path A — API. Paste a key, pick a provider. Show what is and isn't possible with API-only access: behaviour can be measured, internals cannot. **Say that limit plainly on the connect screen**, not buried in docs. This honesty is a feature.

Path B — Open weights. Drag a model folder or paste a HuggingFace ID. Supports up to 1T parameters.

The 1T upload is a real design problem, not a progress bar. A trillion-parameter model is terabytes. The design must:

- Accept a **reference** (S3/GCS URI, HF repo, local mount) as the primary path — uploading through a browser is the fallback, not the default
- Show a **preflight** before any transfer: parameter count, layer count, hidden size, disk required, estimated probe time, estimated cost
- Make the preflight the moment the user decides. Show the cost *before* commitment, never after
- Report progress in **shards completed**, not a percentage — percentages lie on multi-terabyte transfers
- Be resumable and say so
- Verify by hash on arrival and show the hash

5.2 Model shelf

Cards. Model name in monospace, parameter count, when it was last probed, one status dot. Nothing else. A shelf of six models should be readable at a glance from across a desk.

5.3 Run a probe

The user picks **what relation to test**, not "which algorithm to run." Plain language:

- *Which thing caused which*
- *Which came first*
- *Who did it to whom*
- *Who said what* ← flag this one; see §6
- *Part and whole*

Each is a row with a one-line description. Multi-select. One primary button.

Before the run starts, the app runs the pre-flight gate and shows the result. If the stimulus set is surface-solvable, the run is refused with the reason. This is a headline feature: *the tool refuses to run a test that would produce a meaningless number.* Design the refusal as a confident, useful state — not an error.

5.4 The result — Relation Profile

The main screen. A stacked set of Floor components, one per relation, sorted by score. Non-overlapping confidence intervals should be visually obvious without explanation.

Above it: one sentence generated from the data, no adjectives.

> `Qwen2.5-7B-Instruct` binds cause and time strongly. It binds speaker to instruction weakly.

Below it: the claim-state strip — what this run does and does not establish. Always present. Never collapsible-by-default.

5.5 Relation detail

Layer-by-layer curve. The two example sentences shown side by side with the **differing words highlighted and everything else visibly identical** — this is the "aha" moment of the whole product and deserves the most careful typographic treatment on any screen. The user should *see* that the two sentences use the same words.

Then: the exact reproduction command, the result hash, the raw JSON link.

5.6 Report

One page, printable, shareable. Model identity, method, results with floors, claim states, hashes, reproduction steps. It must be credible to a skeptical third party reading it cold, with no access to the app.

---

6. The capability that deserves its own surface

Speaker binding — the prompt-injection precondition.

Measured finding: models bind *cause* and *time* strongly, and bind *who is speaking* far more weakly. Instruction tuning does **not** close that gap. Prompt injection exploits exactly this.

This is the most commercially legible thing the harness does — it produces a **pre-deployment number** for a vulnerability the industry currently measures only by counting successful attacks after the fact.

Give it a dedicated view: the model's speaker-binding score against its own cause/time scores, so the gap is the visual. Do **not** call it a "security score" or attach a letter grade. State what it is and what it is not:

> Measures whether the model's internal state distinguishes who issued an instruction. Does not measure whether a deployed system resists a live attack.

That restraint is the credibility.

---

7. What must never appear

- A single composite "trustworthiness score" — the product's integrity depends on refusing to collapse distinct measurements into one number
- Letter grades, five-star ratings, gauges, speedometers
- Any result shown without its floor
- Rounded metrics
- The words "certified", "guaranteed", "100%", "world-first", "court-admissible"
- Screens that hide the method

---

8. Deliverables requested

1. Connect screen (both paths, one layout)
2. 1T preflight + shard-progress state
3. Model shelf
4. Probe selection, including the **gate-refusal** state
5. Relation Profile (the main result screen)
6. Relation detail with the matched-sentence treatment
7. Speaker-binding view
8. Report page (print layout)
9. Empty, loading, running, refused, and withdrawn-result states
10. Dark and light, both first-class

Test of success: a skeptical ML researcher looks at the Relation Profile screen for five seconds and understands both what was

measured and why the baseline cannot be gamed — without reading a paragraph.

CAS Harness — Design Brief, Part 2 Landing page · Dashboard · The Explainer · Capability surfaces Read Part 1 first. All principles there still bind — especially: **the floor is the hero**, **honest states are first-class**, **numbers exact**.

---

9. The Explainer — the floating AI button

A persistent, movable assistant (Claude via API) that explains what a result means, in the user's own words, wherever they are in the app.

This is the difference between an instrument only a researcher can read and a product an executive can act on. It is also the single easiest place to destroy the product's credibility, so its constraints matter more than its looks.

9.1 Form

- **Resting state:** a `48px` circle. No label, no glow, no pulse, no "AI ✨". A single quiet mark. It should read as a tool, not a mascot.
- **Position:** free. The user drags it anywhere; it magnetises to the nearest edge and stays there. **Position persists per user across sessions and across screens** — if they park it bottom-left, it is bottom-left forever until moved.
- **Out of the way:** dragging it to a screen corner collapses it to a `12px` edge tab — present, nearly invisible, one tap to restore. It must be possible to make this thing effectively disappear without disabling it.
- **Opened:** expands into a panel anchored to its docked edge — a side sheet, not a modal. **The result stays visible while the panel is open.** The user must be able to look at the number and the explanation simultaneously. Never cover the data being discussed.
- **Never:** bounces, wiggles, emits notification dots, or speaks first. It waits.

9.2 Behaviour

- **Context-aware.** It always knows which screen, which model, and which result the user is looking at. Opening it on a Relation Profile pre-loads that run — no "what would you like to know?" cold start.

- **Opens with three suggested questions**, generated from the current screen, not a generic list:
  - *Why does this beat the baseline?*
  - *What could still be wrong with this?*
  - *What does this mean for my deployment?*

  The second one is mandatory on every screen. A product that will argue against its own result is a product people believe.
- **Continuity.** Walking from the Profile to a Relation detail carries the thread. It does not reset.
- **Selection-aware.** Highlight any number anywhere in the app → a small "explain" affordance appears near the selection, routed to the same panel.

9.3 Grounding rules — non-negotiable

The Explainer is **fed the actual result JSON, the claim state, and the method description for the current view.** It answers from that payload.

- It **may** restate, translate to plain language, compare to other runs in the workspace, and explain method.
- It **may not** invent numbers, estimate values not present, or speculate about models not tested.
- Any figure it states must appear in the payload. If asked something the data cannot answer, the correct response is *"this run doesn't measure that"* — and the UI should make that a normal, well-designed response, not a failure.
- **It must carry the claim state.** If a result is `BOUNDED`, the explanation says so unprompted. If `WITHDRAWN`, it leads with that.
- **Every explanation is stamped** with the model that produced it and a link to the underlying JSON. The explanation is commentary; the data is the record. Never blur them.

9.4 Failure states, designed

- No API key → the panel explains what it would do and where to add a key. It does not nag elsewhere.
- API down → the button greys out. **The app is fully usable without it.** The Explainer is never on the critical path to reading a result.

- Streaming → text streams in; the three suggestions stay pinned.

---

10. Landing page

One scroll. Five sections. No carousel, no logo wall, no testimonial slider, no chat bubble.

1. **The claim.** One sentence, large, alone on a near-empty ground:

   > See what a model actually represents — measured against a baseline that cannot be gamed.

   One button: *Connect a model*. Nothing else above the fold.

2. **The proof, not a description of the proof.** A live, static Floor component with real numbers from a real run. The two matched sentences underneath, differing words highlighted, everything else identical. **This is the whole pitch and it is visual, not prose.** A visitor should understand the method here without a paragraph.

3. **What it measures.** Six relation names, one line each. No icons. No cards with drop shadows.

4. **The finding worth the visit.** Speaker binding — models bind cause and time strongly, and who-is-speaking weakly, and instruction tuning does not close the gap. Two Floor components side by side. One sentence.

5. **What it does not claim.** An actual section on the landing page listing the limits. Nobody does this. It will be the most persuasive block on the page, and it costs nothing because the limits are already true.

Footer: method, reproduction, pricing. No newsletter capture.

---

11. Dashboard

The home surface after login. **Not a metrics wall.**

- **Row 1 — Models.** The shelf from Part 1 §5.2. Nothing added.
- **Row 2 — Recent runs.** Model, relation set, verdict, timestamp. One line each, monospace numbers.

- **Row 3 — Attention.** Only appears when something needs a person: a run refused by the pre-flight gate, a result auto-bounded by a later run, a drift alarm. **Empty by default and that is the good state.** Do not manufacture activity to fill it.

No sparkline strip. No "total probes run" vanity counters. No activity heat map.

---

12. Capability surfaces

Each capability gets a surface. **Each surface renders its true claim state** — the statuses below are current and must be shown in the UI, not softened. Designing an unproven capability as though it were proven is the one failure this product cannot survive.

12.1 Cognitive structure detection — `REAL`

The pinned-pair engine. This is the flagship and Part 1 already specifies it (Relation Profile, Relation detail). Everything else on this list is secondary to it.

12.2 Prompt-injection detection — split surface, and be blunt

Two different things share this name and the UI must not merge them.

- **Speaker-binding measurement — `REAL`.** The representational precondition. Part 1 §6.
- **Text-surface injection filter (C11) — `NOT LICENSED for production`.** It is a deterministic text filter. It **provably cannot** separate a legitimate system instruction from an injected user instruction carrying the same words — those inputs are the same word multiset and the same n-gram bag.

Design requirement: show the two side by side with a demonstration — the same pair of sentences, the filter returning identical output for both, the binding measurement separating them. Turning a known weakness into an explicit comparison is stronger than hiding it.

12.3 η→ε bridge (probe → prefix → likelihood) — `REAL`

Read a state from activations, express it as an English prefix, measure the shift in what the model says next. Validated across **6 model points, 3 families, 160M → 20B**; the largest effect `+1.412` at 20B on the 40-Element axis.

The control is the story: a raw-CAS-symbol prefix is null at every scale (`−0.020` **to** `+0.003`). Meaning is doing the work, not the notation. Design it as a three-panel flow — *read → phrase → measure* — with the null control shown as a flat companion bar. Never show the effect without the null beside it.

12.4 Lacunarity — `VALIDATED` as diagnostic · `NULL` as injection detector

A gappiness measure over a state profile. Empirical range `0.793` → `0.0016`. Used as a preflight: a lumpy axis (Noetic-alone `0.793`) predicts steering will fail; a clean axis (Foundation `0.021` , 38× cleaner) predicts it will work.

It failed as an injection detector and the UI must say so: covert-vs-benign `p = 0.37` , overt in the *opposite* direction (`p = 1.3e−5`), AUC `0.33` .

Design as a **go/no-go gauge before steering**, never as a threat indicator.

12.5 Steering — `VALIDATED` (drive-local) · `FALSIFIED` (global)

Drive-local eraser steering works across architectures. **Global steering is falsified** — that is a recorded negative and must remain visible in the UI, not quietly dropped.

Surface: pick a direction, set strength, see the output shift **against a norm-matched random direction** — always. The random control is not optional; a big effect that a random direction also produces is worth nothing. Show both bars, always, side by side.

12.6 probe → ablate → steer loop

The end-to-end arc: find a direction, remove it, watch behaviour move, put it back.

Honest limit for the UI: the attested engine has **no runtime projection/ablation mode** — it adds, it does not project out. So

the app measures **sufficiency** (inject → it moves), not **necessity** (remove → it stops). Label the loop stages accordingly and grey the stage that is not implemented rather than faking it. **Restoration is the receipt** — when ablation does land, re-adding the direction and recovering behaviour is the proof step, and it should be the visual climax of this surface.

12.7 Multi-layer trace + blind-spot detection — `PARTIAL`, controls missing

Classification across layers reaches `95.4%` on a 10-way task at Pythia-160M. **Chance is `10.0%`, and that is the only reference point this result has** — no text baseline, no shuffled-label null, no matched control.

Design requirement: this surface must render the missing controls as visible empty slots, not omit them. A user seeing `95.4%` must also see that three control rows are unfilled. Beating chance is not evidence of anything CAS-specific — a topic classifier beats 10% too.

12.8 Fractal classifier — `n = 1`, not a licensed capability

Classifies an axis as `Stabilizing` / `Oscillatory` / `Amplifying`, which predicts whether steering will succeed. Confirmed instance: Pythia-160M Noetic axis classed `Oscillatory` → Noetic steering subsequently failed, as predicted.

One confirmed prediction is not a prediction record. Show it as a labelled hypothesis with its evidence count visible (`n = 1`). If the count rises, the UI reflects it. Do not present it as a working oracle.

12.9 Layer indexing — `RESOLVED`, infrastructure

`L0` is the **output of block 0**, not the embeddings — which is why a layer-0 result is real signal and not a lookup-table artifact. Not a feature screen. It belongs as a one-line clarification wherever layer indices appear, because the first thing a sharp reviewer will ask about an L0 result is exactly this.

---

13. Additional deliverables

11. Landing page, full scroll

12. Dashboard, populated and empty
13. Explainer: resting, docked-to-edge, collapsed tab, open panel, no-key, offline, streaming
14. Explainer drag-and-dock interaction spec
15. Each capability surface in §12, **rendering its true claim state**
16. The injection split-surface comparison (§12.2)
17. A capability whose status is `NULL` or `FALSIFIED`, designed as carefully as a win — this is the house style test

Test of success for Part 2: a visitor who reads only the landing page understands the method; a customer who never opens the Explainer can still use the product; and a reviewer who opens the weakest capability surface finds its weakness stated before they can discover it themselves.

CAS Harness — Design Brief, Part 3 (for Claude Design) Novel-capability surfaces · floating Guide mechanics · status tokens · build order Read Part 1 and Part 2 first. Every principle there still binds — especially: the floor is the hero, honest states are first-class, numbers exact and monospace, one idea per screen, nothing decorative moves.

Part 3 closes the gaps: the η→ε bridge, lacunarity, steering, and prompt-injection surfaces (the briefs' §12 did not cover them), the full FAB mechanics of the floating Guide, the verdict-colour token table, and the build order.

---

14. Status and verdict tokens — one table, used everywhere

Map the owner's result-marker scale to UI tokens. Marker first in every row; colour only as a 6px left rule or 4px dot. Never a saturated fill.

marker	meaning	UI treatment
✅	CONFIRMED WIN — unpinned-floor pass, survived adversarial verification, licensed wording	calm green dot + exact word, e.g. 'VALIDATED'
🟩	STRONG — real, passed controls, carries a named caveat	green dot, caveat line rendered, not hidden
🟨	PARTIAL — real but bounded	amber dot
🟧	WEAK — marginal; do not build on it	amber-orange dot, desaturated
🟥	FAILED / WITHDRAWN — kept in the record, never deleted	desaturated red, always present on the surface
⬜	NULL / INCONCLUSIVE / NOT RUN	grey outline, no fill
🔧	METHOD/PROCESS WIN — instrument, not science result	neutral dot + 'apparatus' tag
🔒	BLOCKED ON OWNER	neutral outline + owner-action row

Claim-state words ('REAL', 'VALIDATED', 'LICENSED', 'BOUNDED'/'PARTIAL', 'GREY', 'NULL', 'WITHDRAWN', 'ENGINEERED-NOT-VALIDATED', 'NOT RUN') are the ONLY verdict vocabulary. The word 'ACCEPT' appears nowhere in the product. A composite 'trustworthiness score' appears nowhere. Evidence levels (wired / measured / gate-passed) are three distinct rows in the evidence drawer — 'wired' never renders like 'measured'.

---

15. The floating Guide — FAB mechanics (consolidates Part 2 §9)

The Guide is a 48px circle, Claude via API, that explains findings in the user's words and travels with them. Resting: quiet mark, no glow, no label, no mascot.

15.1 Movement contract

- **Free placement.** Pointer-drag anywhere on the viewport. On release it magnetises to the nearest edge (16px inset) and stays.
- **Position persists per user, across screens and sessions.** Parked bottom-left = bottom-left until moved.
- **Out of the way:** drag into any corner → collapses to a 12px edge tab. One tap restores. It must be possible to make it effectively disappear without disabling it.
- **Follows travel, never chases.** It does not animate to the user's scroll; it is viewport-fixed (that is the 'follows you' behaviour). The only motion allowed is the edge-snap spring and the panel slide.
- **Collision avoidance.** When a bottom sheet or the evidence drawer opens, the Guide docks above it, not over it. It never covers a primary action or the active number.

15.2 States

resting → dragging (scale 1.04, no bounce) → docked → collapsed-tab → open-panel (side sheet anchored to its docked edge; the result stays visible — panel max-width 380px, never modal, never covers the data being discussed) → streaming (text streams, the three suggested questions stay pinned) → no-key (explains what it would do and where the key goes; nothing else nags) → offline (greys out; the app is fully usable without it).

15.3 Behaviour (unchanged from Part 2 §9.2–9.4, restated as the contract)

- Context-aware: always knows screen + model + result. Opening on any surface pre-loads that run. Three suggestions, the second of which is always 'What could still be wrong with this?' — mandatory on every screen.
- Selection-aware: select any number → small 'explain' affordance appears beside the selection, routed to the same panel.
- Continuity: the thread follows the user from surface to surface.
- Grounding, non-negotiable: fed only the result JSON, claim state, and method description. May restate, translate, compare within the workspace. May not invent numbers, estimate absent values, or speculate about untested models. Must carry the claim state unprompted (a 'BOUNDED' result is introduced as bounded). 'This run doesn't measure that' is a designed, normal answer. Every explanation is stamped with the producing model and links the underlying JSON — commentary and record are never blurred.

15.4 Accessibility

Keyboard-reachable (Tab order: after primary actions), Enter opens, Esc docks it back. Screen-reader label 'Explain this view'. Reduced-motion: snap and stream become instant. Min touch target 44px on mobile; panel becomes a bottom sheet there.

---

16. Surface — η→ε bridge (eta/epsilon)

What it is: read a CAS class out of the hidden states (η), write it as an English prefix (ε), prepend it to the prompt, and measure how far the next-token likelihood moves toward the CAS-expected continuation versus its involution partner.

Layout — a three-panel arc, read left to right:

1. **η panel** — the probe readout: the class the probe recovered, its confidence, layer. Monospace.
2. **ε panel** — the generated English prefix, shown as the exact string prepended. One line, quoted.

3. **Shift panel** — the Floor component, reused verbatim: bar from the floor (0 = no shift), the probe-minus-partner margin as the measured value, 95% CI bracket.

Under the arc, one monospace line, exact: +1.412050 margin +2.881239 floor 0.000 n=80 seed 20260528

Honest states rendered in place:

- The pre-registered stance-matched non-CAS prefix control was **never run** → its control slot is an **empty slot**, labelled, never omitted. The surface status reads 'REAL' + 'OPEN (missing control)', ceiling 'PARTIAL'.
- The raw-CAS-symbol prefix control row sits directly under the shift panel (null at every scale: −0.020 to +0.003) — it is the 'the notation alone does nothing' row and it makes the English-prefix lift mean something.

Play-out: the bridge is the demo moment. The landing-page strip shows the same three panels with the real 160M numbers; on the surface, toggling ε on/off animates the bar from its floor once, and never again.

---

17. Surface — Lacunarity (factorization-gappiness preflight)

What it is: how gappy or lumpy a candidate axis is inside the activation geometry (Var/Mean^2 across layers). A cheap gate **before** spending GPU time. Pass threshold ≤ 0.05.

Layout: one horizontal band, log-annotated, spanning the empirical range 0.793 → 0.0016, with the **0.05 gate as the hero line** (the Floor pattern, reused for a threshold instead of a baseline). The measured axis is a single marker with its exact value in monospace. Under it, one line each:

- Foundation axes: 0.006715 (20B F28), 0.021 (160M), 0.015 (1B), 0.0084 (Qwen 1.5B) → all left of the gate, drawn calm
- Noetic-alone 0.793 → right of the gate, drawn as the named fail signature: 'do not proceed to steering on a lumpy axis'

Two honesty rows, both first-class:

- As a steering preflight: 'VALIDATED' apparatus.
- As a prompt-injection / structural-break detector: 'NULL' — it failed the benign-append control in both forms (AUC 0.33,

wrong direction on overt). This is the house-style test: a negative designed as carefully as a win.

---

18. Surface — Prompt-injection detection

What it is (and is not): a deterministic text-surface scanner — raw, zero-width-stripped, HTML-entity-decoded, URL-decoded, base64 layers — over prompts, plus the speaker-binding measurement from Part 2 §12.2. It is **not** a hidden-state detector; it is **not** a live-attack guarantee.

Layout — the scanner view: input prompt rendered as a layer stack. Each decoded layer is a row; anything the layers disagree on is highlighted once, quietly. Verdict chip per payload: 'INJECTED' / 'BENIGN' / 'INCONCLUSIVE', with the exact rule that fired. Below, the speaker-binding split-surface: cause-binding and time-binding strong, speaker-binding weak, the **gap as the visual**, one sentence of scope:

> Measures whether the model's internal state distinguishes who issued an instruction. Does not measure whether a deployed system resists a live attack.

No security score, no letter grade.

---

19. Surface — Steering (eraser channel + additive dial)

What it is: the only generation-steering that worked (drive-local eraser channel, 'VALIDATED'/'LICENSED' bounded to drives) plus the additive direction path (research-grade 'EXPLORATORY_POSITIVE', failed its own P2 bar — shown as such).

Layout — before/after, with the control between them:

- Two generation panels side by side, monospace completions, the changed span underlined once.
- Between them, the intervention: a **slider with an exact numeric readout** (never a gauge, never a speedometer). Amplification ×2.0, polarity flip ×−1.0, eraser-channel suppress — each is one slider, one line of scope.

- Under both, the matched control row: norm-matched random direction, its own before/after, so 'big effect' is never shown without 'random same-size effect'.

Labelling discipline: the engine adds; it does not project out. **Sufficiency** stages are solid; the **necessity** stage (true ablation) is greyed until the attested engine implements it. **Restoration is the receipt:** re-adding the direction and recovering behaviour is the visual climax — one full-strength highlight, once.

---

20. Surface — probe → ablate → steer loop (completes Part 2 §12.6)

One stage ladder, five rungs: **Probe** → **Ablate** → **Patch-back** → **Steer** → **Restore.** Each rung is a small Floor row (its own floor, its own control). The rung that is not yet implemented is greyed with the word 'NOT RUN' — never faked, never skipped. The ladder reads as one causal sentence, top to bottom, with the matched-random control track running alongside every rung. This surface is the product's argument structure made visible.

---

21. Surfaces carried from Parts 1–2 (index, for the build list)

- Cognitive structure detection ('REAL') — the pinned-pair engine, flagship (Part 1, Part 2 §12.1)
- Speaker binding / injection precondition — dedicated view (Part 1 §6)
- Injection split-surface (Part 2 §12.2)
- Multi-layer trace + blind-spot ('PARTIAL') — missing controls render as **empty slots** (Part 2 §12.7)
- Fractal classifier ('n = 1') — shown as a labelled hypothesis with its evidence count visible (Part 2 §12.8)
- Layer indexing ('RESOLVED') — the one-line clarification wherever layer indices appear: L0 = output of block 0, not embeddings (Part 2 §12.9)
- η→ε bridge, Lacunarity, Injection scanner, Steering, probe→ablate→steer — **this brief, §16–§20**

---

22. Model intake (reference: Part 1 §5.1) — one addition

The open-weight path (up to 1T params) already carries reference-first intake, preflight-before-cost, shard-count progress, resumability, and hash-on-arrival. Add the **attestation states**, because pinning is the trust moment:

- 'PINNED' — model ID + HF revision + tree-manifest sha256, rendered monospace, copyable
- 'ATTESTING…' — byte-verification progress against the pin (real stages: config → weights → tokenizer → manifest; never a fake percent)
- 'FROZEN' — the locked state before a run; the lock visual is the whole brand
- 'RE-PIN REQUIRED' — the designed state when a pin and the verifier disagree (owner-approved re-pin, never silent)

API-connected models carry a one-line scope on the connect screen: behaviour can be measured, internals cannot — unless open weights are provided. The line is on the screen, not in the docs.

---

23. Landing-page addition — the 'See it work' strip

One quiet strip between 'The proof' and 'What it measures' (Part 2 §10): the capability names as a single line of links — η→ε bridge · lacunarity · speaker binding · injection scanner · steering · fractal · layer trace — each with its status dot. No cards, no icons, no hover explosions. A visitor reads the whole surface in one glance and understands the house style before clicking anywhere.

---

24. Component inventory and build order

P0 (the pitch + the read path): Floor component (with threshold variant) · status/verdict token system · landing page incl. bridge strip · dashboard (populated + empty) · connect screen (API + open-weight preflight) · model shelf · cognitive-structure surface · speaker-binding surface · bridge surface.

P1 (the trust path): evidence drawer (six proof chips, five-check verify, hash chain, WORM anchor line) · findings feed with marker glyphs · injection scanner · lacunarity · steering · probe→ablate→steer ladder · layer-trace surface with empty control slots · fractal surface (n=1) · report page (print) · dark/light parity.

P2 (the Guide): FAB + drag/dock/persistence · side-sheet panel · context payloads per surface · selection-aware explain · no-key / offline / streaming states · a11y and reduced-motion pass.

House-style acceptance tests (from Parts 1–2, restated):

1. A skeptical researcher reads the Relation Profile in five seconds and understands what was measured and why the floor cannot be gamed — without a paragraph.
2. A visitor who reads only the landing page understands the method.
3. A customer who never opens the Guide can still use the product.
4. A reviewer who opens the weakest surface finds its weakness stated before they discover it themselves.
5. No 'ACCEPT', no composite score, no rounded number, no gradient hero, no decorative motion — anywhere.